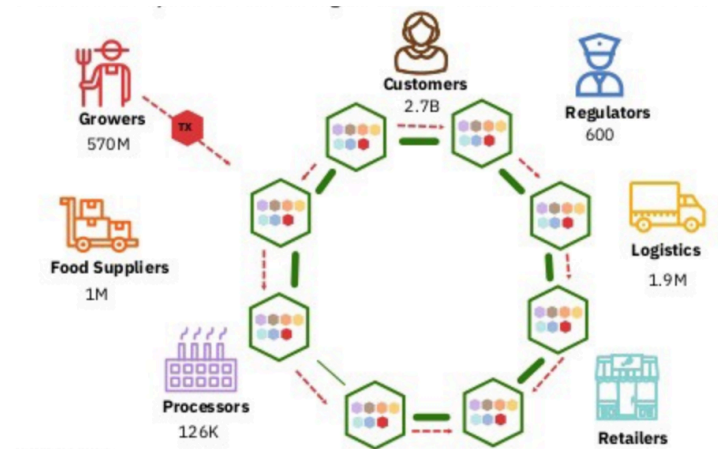




Food Chain

Vytvořte aplikaci, která zjednodušeně implementuje blockchain a nad ním systém pro sledování toku potravin od jejich vypěstování, přes procesování, skladování, distribuci až po prodej zákazníkovi. Součástí ekosystému jsou entity jako zemědělec/farmář, zpracovatel, sklad, prodejce, distributor, zákazník, které tvoří síť bodů, přes které potraviny procházejí (např. potravina, surovina). Cílem ekosystému je zajistit, aby v každém bodě systému bylo možné dohledat, kterými body potravina prošla, co se s ní v každém bodě dělo, a zároveň aby nebylo možné tyto informace zpětně manipulovat. Tím se zabrání situacím, kdy je například maso z Polska vydáváno s certifikátem masa z Rakouska, nebo je upravována doba a teplota skladování.



Součástí aplikace je i předpřipravená konfigurace potravin a producentů/konzumentů, na které jsou ukázány implementované požadavky.



Funkční požadavky

Povinné

1. Hlavní entity: zemědělec/farmář, zpracovatel, sklad, prodejce, distributor, zákazník, potravina.
2. Jednotlivé strany (parties) si v ekosystému předávají potraviny a za operace s nimi si účtují peníze. Každá strana s potravinou provádí určité operace. Každá operace má parametry – například skladování: délka 4 dny, teplota 12 °C, vlhkost atd. Operace provedená s konkrétními parametry a konkrétní stranou = transakce.
3. Systém je realizován pomocí zjednodušené blockchain platformy. Každá transakce s parametry a identifikací strany, která ji provedla, je zaznamenána a odkazuje se na předchozí transakci. Již provedené transakce a jejich parametry nelze zpětně upravovat.
4. V každém bodě životního cyklu potraviny lze získat věrohodné informace o tom, přes jaké strany potravina prošla a jaké operace s ní byly provedeny. Systém umožňuje tyto informace generovat ve formě reportu (textový soubor).
5. Systém realizuje tzv. kanály. Kanál spojuje strany, operace a má svůj vlastní řetězec událostí. Příkladem je kanál pro výměnu zeleniny, kanál pro maso, kanál pro platby. Každá operace má určeno, v jakém kanálu se může provést, stejně tak strana má definováno, v jakých kanálech je účastníkem.
6. Systém detekuje tzv. double spending problém. Tuto detekci simulujte na zvoleném scénáři, například když se zpracovatel pokusí prodat stejnou potravinu dvakrát pod stejným certifikátem (dokládajícím původ potraviny).
7. Systém detekuje pokus o zpětnou změnu v řetězci událostí. Tuto detekci simulujte na zvoleném scénáři.
8. Ze systému lze generovat následující reporty:

- **PartiesReport** udává jaké strany se podílely na zpracování daných potravin, jak dlouho se u nich potraviny zdržely, jakou marži si strana účtovala na vstupní cenu.
- **FoodChainReport** pro každou potravinu vypíše, přes jaké strany prošla a jaké operace s jakými parametry u ní byly provedeny.
- **SecurityReport** uvádí jaké strany se pokusily podvrhnout původ potravin nebo provést double spending a kolikrát.
- **TransactionReport** pro každý diskretní krok ekosystému vypíše transakce, které byly provedeny, kým a kolik měla každá strana v ekosystému peněz a potravin.

9. Ukažte názorně běh aplikace na příkladu jednoho ekosystému o alespoň 12 stranách a 5 potravinách. Simulaci proveďte pomocí diskretní simulace, kde každý krok odpovídá jednomu bloku:

- Diskretní krok 0:
 - Strana A vyšle požadavek
 - Strana B reaguje na požadavek strany A předáním potraviny straně C
 - Strana D předává potravinu straně C a zároveň vysílá požadavek
 - Strana E reaguje jako první na požadavek strany D a začíná zpracovávat potravinu
- Diskretní krok 1:
 - Strana C předává potravinu straně A
 - Strana E předává potravinu straně C
 - Strana E předává potravinu straně D ⇒ detekován double spending problém ... atd

Bonusové

1. Do kanálů lze rozesílat požadavky, které obdrží strany účastníci se daných kanálů. Například požadavek „Chtěl bych 150 kg masa“. Strany na požadavky mohou, ale nemusí reagovat. Strany se mohou registrovat/odregistrovat buď do/z celého kanálu nebo na typ požadavku v rámci kanálu.
2. Zpracování potraviny u strany je realizováno stavovým automatem – potravina prochází různými stavy, než může být předána další straně. Mezi jednotlivými diskretními kroky simulace může dojít pouze k jednomu přechodu mezi stavy.



Nefunkční požadavky

- Není požadována autentizace ani autorizace.
- Aplikace nemá grafické rozhraní. Komunikuje s uživatelem pomocí příkazové řádky nebo výpisů do souboru.
- V systému neexistuje centrální autorita, přes kterou by se prováděly transakce. Aplikace běží pouze v jedné JVM, není potřeba distribuovaná aplikace. Není nutné řešit více vláken – stačí jedno, kde se postupně aplikují všechny transakce provedené jednotlivými stranami v rámci aktuálního kroku diskretní simulace.
- Pište aplikaci tak, aby byly dobře skryté metody a proměnné, které nemají být dostupné ostatním třídám. Vygenerovaný Javadoc by měl obsahovat co nejméně public metod a proměnných.
- Namísto PKI infrastruktury, veřejných a soukromých klíčů, šifrování/dešifrování pracujte pouze s abstrakcí – například jednoduchá Java třída reprezentující klíč (PRK, PUK) uživatele, metoda, která podepíše obsah – pouze nastaví na obsahu příznak podepsáno a jakým klíčem.
- Reporty jsou generovány do textového souboru.

- Konfigurace ekosystému může být vytvořena přímo v kódu nebo externím souboru (např. JSON).

Vhodné design patterny

- Factory / Factory Method
- Builder
- Chain of Responsibility
- Memento
- Observer
- Visitor
- State
- Monáda (2 body místo jednoho za implementaci vlastní monády)
- Streamy

Doporučená literatura

- [https://www.geeksforgeeks.org/dsa/introduction-to-merkle-tree/\[1\]](https://www.geeksforgeeks.org/dsa/introduction-to-merkle-tree/[1])
- [https://www.theblock.co/learn/245697/what-are-blocks-in-a-blockchain\[2\]](https://www.theblock.co/learn/245697/what-are-blocks-in-a-blockchain[2])

Hypertext Links

[1] <https://www.geeksforgeeks.org/dsa/introduction-to-merkle-tree/>

[2] <https://www.theblock.co/learn/245697/what-are-blocks-in-a-blockchain>

courses/b6b36omo/sem/food_chain.txt · Last modified: 2025/09/20 21:10 by jarymiro

Copyright © 2025 CTU in Prague | Operated by [IT Center of Faculty of Electrical Engineering](#) | Bug reports and suggestions [ServiceDesk CTU](#)